

Examen Archi 3A – 2020/2021

November 28, 2022

1 Optimisation microarchitecturale via le renommage de registres

On se place dans le contexte d'un jeu d'instruction définissant 31 registres généralistes 64-bit ($R1$ à $R31$). Le jeu d'instruction définit de plus un registre 64-bit valant toujours $0x0$, $R0$.

On possède un processeur implémentant cette architecture via une microarchitecture à exécution dans le désordre. Les structures nécessaires à l'exécution correcte et efficace des instructions sont donc déjà présentes : *Rename Map Table*, *Scoreboard*, *Active List*, *Free List*, *Physical Register File* (fichier de registres) *Reorder Buffer*, *Instruction Queue* (ordonnanceur) et *Load/Store Queue*.

Dans cette microarchitecture, le fichier de registres possède 127 registres physiques $p1$ à $p127$, auquel on ajoute un unique registre câblé à $0x0$, $p0$ afin d'implémenter le registre logique $R0$. En particulier, une instruction utilisant $R0$ comme opérande (source ou destination) renommerait toujours $R0$ par $p0$. Cela signifie notamment que $p0$ ne fait pas partie de la *Free List*, et que le registre $p0$ est toujours prêt dans le *Scoreboard*, car sa valeur courante est toujours $0x0$.

On souhaite améliorer la performance de la microarchitecture existante en tirant parti du renommage de registre. En particulier, on remarque qu'il est possible de calculer le résultat de certaines instructions sans les exécuter, mais en manipulant l'association des registres physiques aux registres logiques.

1.1 “Zero-idiom Elimination”

Dans un premier temps, on remarque que les instructions pour lesquelles on peut prouver que le résultat généré vaut $0x0$ simplement via l'instruction et le nom de ses opérandes (registres architecturaux) peut “s'éliminer” (i.e., n'a pas besoin d'être ordonnancée et exécutée) à l'étage de renommage en renommant le registre logique de destination par $p0$. Quelques exemples où renommer $R1$ par $p0$ permet d'exécuter l'instruction au renommage :

```
xor  R1, R2, R2    # R1 = R2 xor R2 = 0x0
and  R1, R2, R0    # R1 = R2 and 0x0 = 0x0
or   R1, R0, R0    # R1 = 0x0 or 0x0 = 0x0
```

Question 1.1 : Quel(s) avantage(s) aura-t-on à éliminer les instructions que l'on peut prouver comme produisant toujours $0x0$ dans l'étage de renommage ?

Question 1.2 : Est-ce qu'une instruction éliminée peut complètement disparaître de toutes les structures du pipeline au renommage ? On pourra par exemple considérer la suite d'instructions suivante :

```

A:
  add  R2, R4, R5      # R2 = R4 + R5
  cbnz R2, B           # Saut vers B si R2 != 0
  xor  R3, R2, R2      # R3 = R2 xor R2 = 0x0; elim
  ...
B:
  sub  R1, R2, R3      # R1 = R2 + R3

```

Est-il alors possible de faire complètement disparaître *xor R1, R2, R2* du pipeline au renommage ?

Question 1.3 : Est-ce qu’implémenter cette élimination dans notre microarchitecture requiert un support matériel particulier (par ex., nouvelles structures, nouveaux champs dans des structures existantes, nouveaux circuits, etc.) ? On pourra notamment chercher à répondre aux questions suivantes ¹ :

- Quel impact sur l’algorithme assurant le recyclage des registres physiques ?
- Quel impact sur la réparation des structures de l’étage de renommage lors d’un vidage de pipeline ?

1.2 “Move Elimination”

Dans un second temps, on remarque que les instructions qui ne font que copier la valeur d’un registre dans un autre (instruction *move*) peuvent aussi s’éliminer à l’étage de renommage, en renommant le registre logique de destination par le registre physique qui correspond au registre logique source. Par exemple :

```

move R1(p12), R2(p42) # p12 = p42
move R3(p18), R1(p12) # p18 = p12

```

devient :

```

move R1(p42), R2(p42) # R1 = R2; elim
move R3(p42), R1(p42) # R3 = R1; elim

```

Question 2.1 : Quel(s) avantage(s) aura-t-on à éliminer les instructions *move* dans l’étage de renommage ?

Question 2.2 : Est-ce qu’implémenter cette élimination dans notre microarchitecture requiert un support matériel particulier (par ex., nouvelles structures, nouveaux champs dans des structures existantes, nouveaux circuits, etc.) ? On pourra notamment chercher à répondre aux questions suivantes² :

- Quand le résultat d’une instruction *move* éliminée devient-il prêt ? On pourra par exemple considérer les instructions suivantes :

```

add  R2(p42), R8(p127), R9(p1) # R2 = R8 + R9
move R1(p42), R2(p42)          # R1 = R2; elim
add  R4(p12), R1(p42), R1(p42) # R4 = R1 + R1

```

Peut-on alors exécuter *add R4, R1, R1* immédiatement après son entrée dans l’ordonnanceur, puisque *move R1, R2* sera considéré comme exécuté (car éliminé au renommage) et produit R1 lu par *add R4, R1, R1* ?

¹Les questions sont données à titre indicatif afin de guider votre réflexion. L’objectif est cependant de fournir une réponse à la Question 1.3 dans son entièreté.

²Les questions et exemples sont données à titre indicatif afin de guider votre réflexion, et devraient tous être lus avant de tenter de répondre à la question 2.2 dans son entièreté.

- Quand est-il possible de remettre le registre physique de destination d'une instruction *move* éliminée dans la *Free List* ? Il faudra notamment (mais pas seulement) considérer le cas où plusieurs définitions d'un même registre architectural utilisent le même registre physique (ici *R2* est renommé en *p42* par le sub, et renommé en *p42* par le dernier move, qui est éliminé) :

```
sub  R2(p42), R1(p18), R8(p13) # R2 = R1 + R8
move R1(p42), R2(p42)         # R1 = R2; elim
move R3(p42), R1(p42)         # R3 = R1; elim
move R2(p42), R3(p42)         # R2 = R3; elim
```

- Quel impact sur la réparation des structures de l'étage de renommage lors d'un vidage de pipeline ? On pourra par exemple considérer les instructions suivantes :

```
A:
  add  R2, R4, R5      # R2 = R4 + R5
  cbnz R2, B          # Saut vers B si R2 == 0
  move R1, R2          # R1 = R2; elim
  ...
B:
  sub  R1, R2, R3      # R1 = R2 + R3
```

Est-il sûr de remettre le registre physique de destination alloué à R1 pour *move R1, R2* (qui est donc le même que celui alloué à R2 pour *add R2, R4, R5* car le move est éliminé) si on a renommé l'instruction sur le mauvais chemin et que le branchement a sauté vers B ?